

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto "AtomReactChamber"

Curso: *Base de Datos II*

Docente: Ing. Patrick Cuadros Quiroga

Integrante:

Laos González, Adriana Rafaela (2023077474)

Tacna – Perú
2026

Sistema “AtomReactChamber”
Manual Técnico

Versión *1.0*

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Adriana Laos	Ing. Patrick Cuadros Quiroga (pendiente)	(pendiente)	2026-11-07	Primera versión: instalación, configuración, despliegue y solución de problemas

ÍNDICE GENERAL

1. Introducción.....5

1.1 Componentes.....5

2. Instalación local.....5

2.1 Requisitos.....5

2.2 Pasos.....5

2.3 Comandos del monorepo.....6

3. Variables de entorno.....6

3.1 API (backend/.env).....6

3.2 Frontend (tiempo de *build*).....7

3.3 Integraciones.....7

4. Base de datos y Liquibase.....7

4.1 Estructura.....7

4.2 Operaciones frecuentes.....7

4.3 Agregar un cambio de esquema.....8

4.4 Cambiar contenido (niveles, pistas, glosario).....8

4.5 Respaldo y restauración.....8

5. Despliegue en la nube (Terraform + Neon + Render).....8

5.1 Arquitectura de producción.....8

5.2 Primer despliegue.....8

5.3 Publicar una versión.....9

5.4 Migrar a Azure o AWS.....9

6. Secrets y variables de GitHub.....10

7. Automatizaciones.....10

8. Integraciones..... 11

9. Pruebas..... 11

10. Solución de problemas..... 11

11. Seguridad operativa.....12

1. Introducción

Este manual explica cómo instalar, configurar, migrar, probar, desplegar y operar **AtomReactChamber**. Está dirigido a quien mantenga el sistema (administradora, docente del curso o futuros desarrolladores). La arquitectura se describe en el **FD04**; los objetos de la base de datos, en el **diccionario de datos**; y las convenciones de código, en el **estándar de programación**.

1.1 Componentes

Componente	Carpeta	Tecnología	Producción
Frontend	`frontend/`	React 18 + Vite + TypeScript, Tailwind, Canvas 2D	Render Static Site; demo en GitHub Pages (`/app/`)
API	`backend/`	Node 22 + Fastify 5 + TypeScript + `pg`	Render Web Service (plan Free)
Base de datos	`db/`	PostgreSQL 16 + Liquibase (SQL formateado)	Neon (plan Free)
Infraestructura	`infra/terraform/`	Terraform: módulos `database-neon` y `app-render`, entorno `envs/prod`	Estado en HCP Terraform
Automatizaciones	`.github/workflows/`	9 *workflows* de GitHub Actions	GitHub
Integraciones	`extensions/vscode`, `integrations/mcp`, `.claude/`	TypeScript, SDK MCP, Bash	Local / Releases

2. Instalación local

2.1 Requisitos

Herramienta	Versión	Uso
Node.js	≥ 22	Monorepo con *workspaces* npm
PostgreSQL	16	Base de datos local
Liquibase	4.x (requiere Java 17+) **o** Docker	Migraciones (`.github/scripts/liquibase.sh` usa la imagen `liquibase/liquibase:4.33`)
Terraform	≥ 1.9 (opcional)	Infraestructura
Git	—	Control de versiones

2.2 Pasos

```
git clone https://github.com/laos19/demov0.1atomreactchamber.git
cd demov0.1atomreactchamber
npm install                                   # instala frontend,
backend, extensión y MCP

# 1) Base de datos
createdb arc
liquibase --changelog-file=db/changelog/db.changelog-master.yaml \
  --url=jdbc:postgresql://localhost:5432/arc --username=postgres --
password=postgres update
#    (alternativa con Docker)
LIQUIBASE_COMMAND_URL=jdbc:postgresql://localhost:5432/arc \
LIQUIBASE_COMMAND_USERNAME=postgres
```

```
LIQUIBASE_COMMAND_PASSWORD=postgres \
  .github/scripts/liquibase.sh update

# 2) API
cp backend/.env.example backend/.env          # editar valores
(sección 3)
set -a; . backend/.env; set +a                # la API lee
process.env (no carga .env sola)
npm run dev:api                               #
http://localhost:8080 · Swagger en /api/docs

# 3) Frontend
VITE_API_URL=http://localhost:8080 npm run dev   #
http://localhost:5173
```

Modo demostración: `npm run dev` sin `VITE_API_URL` abre la aplicación sin servidor (contenido incluido en el *bundle* y progreso en `localStorage`).

Cuentas de demostración (contexto Liquibase `demo`, `902-datos-demo.sql`): `admin@demo.arc.pe`, `docente@demo.arc.pe`, `estudiante@demo.arc.pe` y `e01…e12@demo.arc.pe`, todas con la contraseña `quimica2026`; aula `QUIM3A`. En producción se excluyen con `--contexts=!demo` si no se desean.

2.3 Comandos del monorepo

Comando	Qué hace
`npm run build`	Compila todos los paquetes
`npm test`	Pruebas de todos los paquetes (el backend requiere `DATABASE_URL`)
`npm run e2e -w frontend`	Playwright + axe sobre el *build* en modo demo
`npm run seed:generate`	Regenera las semillas SQL (900–902) y el contenido del frontend desde `db/data/*.json`
`npm run docs:readme` · `docs:diagrams` · `docs:api`	Secciones del README, diagramas Mermaid y TypeDoc (`site/api`)
`npm run openapi -w backend`	Exporta el contrato OpenAPI

3. Variables de entorno

3.1 API (`backend/.env`)

Variable	Obligatoria	Por defecto	Descripción
`DATABASE_URL`	Sí	—	`postgres://usuario:clave@host:5432/db`. En Neon usar la cadena con `sslmode=require`
`JWT_SECRET`	Sí	—	Mínimo 32 caracteres; en Render lo genera el servicio (`generate_value`)
`CORS_ORIGIN`	No	`*`	Orígenes permitidos separados por comas (p. ej., URL del sitio)
`DOCENTE_CODIGO`	No	`UPT2026`	Código de invitación para registrarse como docente. **Cámbielo en producción**

Variable	Obligatoria	Por defecto	Descripción
`PORT`	No	`8080`	Puerto HTTP
`HOST`	No	`0.0.0.0`	Interfaz de escucha
`NODE_ENV`	No	`development`	`development`, `test` o `production`
`DB_SSL`	No	`auto`	`auto` (TLS si el host no es local), `require` o `disable`

La configuración se valida al iniciar (zod): si falta algo, la API no arranca y muestra qué variable es inválida.

3.2 Frontend (tiempo de *build*)

Variable	Descripción
`VITE_API_URL`	URL base de la API (sin `/api`). Vacía ⇒ modo demostración

3.3 Integraciones

Variable	Usada por	Descripción
`ARC_API_URL`	Servidor MCP, *skill*	URL de la API
`ARC_TOKEN`	Servidor MCP	JWT docente para `resumen_dashboard`
`ARC_EMAIL`, `ARC_PASSWORD`	*Skill* (`arc.sh dashboard`)	Credenciales docentes
`atomreactchamber.apiUrl`, `atomreactchamber.webUrl`	Extensión VS Code (ajustes)	URL de la API y del sitio

4. Base de datos y Liquibase

4.1 Estructura

`db/changelog/db.changelog-master.yaml` incluye 13 archivos (48 *changesets*): `001-extensiones` → `002-seguridad` → `003-catalogo` → `004-juego` → `005-progreso` → `006-telemetria` → `007-funciones` → `008-triggers` → `009-vistas` → `010-indices-fk` → `900-semilla-catalogo` → `901-semilla-juego` → `902-datos-demo` (contexto `demo`).

4.2 Operaciones frecuentes

Tarea	Comando
Ver pendientes	`liquibase status --verbose`
Ver el SQL que se aplicará	`liquibase update-sql`
Aplicar	`liquibase update`
Revertir el último *changeset*	`liquibase rollback-count 1`
Probar todos los *rollback*	`liquibase update-testing-rollback` (en una BD limpia)
Historial	`liquibase history`
Regenerar el diccionario	`psql "$DATABASE_URL" -X -q -A -t -f docs/database/generar-diccionario.sql > docs/database/diccionario-de-datos.md`

4.3 Agregar un cambio de esquema

1. Crear `db/changelog/sql/0NN-<tema>.sql` con `--liquibase formatted sql`.
2. Cada *changeset*: `--changeset adriana.laos:0NN-n comment:…`, `COMMENT ON` para objetos nuevos y `--rollback` obligatorio.
3. Incluirlo en `db.changelog-master.yaml` **antes** de las semillas 9xx.
4. Probar: `update`, `rollback-count 1`, `update`. Ejecutar `npm test -w backend`.
5. Abrir un PR de tipo `db:`; `db.yml` lo valida en una BD efímera.

4.4 Cambiar contenido (niveles, pistas, glosario)

Editar `db/data/juego.json` o `db/data/catalogo.json`, ejecutar `npm run seed:generate`, revisar el *diff* de `900/901-*.sql` y de `frontend/src/data/contenido.generated.json`, y comprobar con `npm test -w frontend` que el *solver* sigue completando los niveles. Las semillas son `runOnChange`, así que el siguiente `liquibase update` publica el contenido.

4.5 Respaldo y restauración

`backup.yml` ejecuta cada día `pg_dump -Fc`, calcula SHA-256, cifra con AES-256 (`BACKUP_PASSPHRASE`) y verifica la restauración comparando filas por tabla. Retención: 7 diarias, 4 semanales, 3 mensuales; además, Neon ofrece recuperación a un punto en el tiempo de 6 horas en el plan Free.

```
gpg --decrypt atomreactchamber-AAAAMMDD-HHMM.dump.gpg >
respaldo.dump   # si está cifrado
sha256sum -c atomreactchamber-AAAAMMDD-HHMM.dump.sha256
pg_restore --no-owner --no-privileges --clean --if-exists -d
"$DATABASE_URL" respaldo.dump
```

5. Despliegue en la nube (Terraform + Neon + Render)

5.1 Arquitectura de producción

Capa	Servicio	Recurso Terraform
Base de datos	Neon PostgreSQL 16 (Free)	`neon_project` (rama `main`, BD `atomreactchamber`, rol `app_owner`)
API	Render Web Service (Free)	`render_web_service.api` (health check `/api/health`)
Frontend	Render Static Site	`render_static_site.web`
Documentación y demo	GitHub Pages	`pages.yml`
Estado de Terraform	HCP Terraform (Free)	bloque `cloud {}` (alternativa: backend `pg`)

Los servicios de Render se crean con despliegue automático desactivado: `release.yml` aplica primero Liquibase y después llama a los *deploy hooks*, de modo que la API nunca arranca con un esquema viejo.

5.2 Primer despliegue

1. **Crear cuentas** (sin tarjeta en planes gratuitos; verifíquelo al registrarse):

   - Neon → *Account settings* → *API keys* → `NEON_API_KEY`.
   - Render (con GitHub, autorizando la GitHub App sobre el repositorio) → *Account settings* → *API Keys* → `RENDER_API_KEY`; el *Owner ID* (`usr-…`/`tea-…`) → `RENDER_OWNER_ID`.
   - HCP Terraform → organización + workspace CLI-driven `atomreactchamber-prod` en modo de ejecución **Local** → token `TF_API_TOKEN`.
   - (Opcional) Infracost → `INFRACOST_API_KEY`.
2. **Cargar secrets y variables** en GitHub (sección 6).
3. **Aplicar Terraform:** *Actions* → *Infraestructura* → *Run workflow* → *action = apply* (o en local):

   ```
   cd infra/terraform/envs/prod
   export NEON_API_KEY=… RENDER_API_KEY=… RENDER_OWNER_ID=…
   TF_CLOUD_ORGANIZATION=mi-org
   terraform login && terraform init && terraform plan -out=tfplan
   && terraform apply tfplan
   terraform output api_url web_url database_jdbc_url
   database_user
   terraform output -raw database_connection_string
   ```
4. Con las salidas, cargar `LIQUIBASE_URL`, `LIQUIBASE_USER`, `LIQUIBASE_PASSWORD`, `DATABASE_URL`, `API_URL` y `WEB_URL`; copiar los *deploy hooks* de cada servicio de Render (`RENDER_DEPLOY_HOOK_API`, `RENDER_DEPLOY_HOOK_WEB`).
5. Hacer *push* a `main`: `db.yml` migra Neon y `release.yml` despliega y ejecuta el *smoke test*.
6. Activar GitHub Pages: *Settings* → *Pages* → *Source: GitHub Actions*; y permitir a Actions crear PR (*Settings* → *Actions* → *General*), requerido por release-please.

5.3 Publicar una versión

Los commits siguen Conventional Commits (`feat:`, `fix:`, `db:`…). release-please mantiene un PR `chore: release x.y.z`; al fusionarlo se crea el tag `vX.Y.Z`, el Release con los artefactos (`.zip` del frontend y backend, `.vsix`, `SHA256SUMS.txt`) y se despliega.

5.4 Migrar a Azure o AWS

`envs/prod` solo depende de las salidas de los módulos (`connection_uri*`, `database_host`, `jdbc_url`, `api_url`, `web_url`). Basta escribir módulos equivalentes (p. ej., `azurerm_postgresql_flexible_server` o `aws_db_instance`) y cambiar el `source`; los datos se trasladan con `pg_dump`/`pg_restore`. La referencia de costos está en `infra/terraform/cost-reference/` (≈ USD 38/mes en AWS o Azure frente a USD 0 actual).

6. Secrets y variables de GitHub

Settings → Secrets and variables → Actions. Todos los *workflows* terminan en verde aunque falten: los pasos que dependen de ellos se omiten y el reporte indica `omitido: falta secret X`.

Secret	Usado en	Valor
`NEON_API_KEY`	`infra.yml`	API key de Neon
`RENDER_API_KEY`	`infra.yml`, `release.yml`	API key de Render
`RENDER_OWNER_ID`	`infra.yml`	`usr-…` o `tea-…`
`TF_API_TOKEN`	`infra.yml`	Token de HCP Terraform
`TF_STATE_PG_CONN_STR`	`infra.yml`	(Alternativa a HCP) conexión de la BD de estado
`INFRACOST_API_KEY`	`infra.yml`	API key de Infracost
`LIQUIBASE_URL`	`db.yml`, `release.yml`	`jdbc:postgresql://…neon.tech:5432/atomreactchamber?sslmode=require`
`LIQUIBASE_USER`	`db.yml`, `release.yml`	`app_owner`
`LIQUIBASE_PASSWORD`	`db.yml`, `release.yml`	Contraseña del rol
`DATABASE_URL`	`backup.yml`	Conexión directa (sin `-pooler`)
`BACKUP_PASSPHRASE`	`backup.yml`	Frase para cifrar respaldos; sin ella el volcado se verifica pero no se sube
`RENDER_DEPLOY_HOOK_API`	`release.yml`	*Deploy hook* de la API
`RENDER_DEPLOY_HOOK_WEB`	`release.yml`	*Deploy hook* del sitio

Variable	Usada en	Valor
`TF_CLOUD_ORGANIZATION`	`infra.yml`	Organización de HCP Terraform
`TF_WORKSPACE`	`infra.yml`	(Opcional) `atomreactchamber-prod`
`API_URL`	`release.yml`, `salud.yml`	URL de la API
`WEB_URL`	`release.yml`	URL del sitio
`BACKUP_RELEASE`	`backup.yml`	`true` para adjuntar respaldos semanales/mensuales a un Release

7. Automatizaciones

Workflow	Disparador	Qué hace
`ci.yml`	PR y *push* a ramas	*Lint*, *typecheck*, actionlint, `terraform fmt`, pruebas
`infra.yml`	*Push*/PR a `main`; manual	Calidad IaC, pruebas (PostgreSQL + Liquibase + backend + frontend + e2e), costos, `plan`/`apply`
`db.yml`	Cambios en `db/**`; manual	Liquibase en BD efímera (incluye prueba de *rollback*) y en producción
`backup.yml`	Diario 03:17 (Lima); manual	Respaldo cifrado y verificado
`release.yml`	*Push* a `main`, tags `v*`	Versión, artefactos, Liquibase, *deploy hooks*, *smoke test*

Workflow	Disparador	Qué hace
`docs.yml`	*Push* a `main`	tbls, diagramas y README generado
`pages.yml`	*Push* a `main`; tras `infra/backup`	Sitio de documentación y demo
`codeql.yml`	*Push*/PR, semanal	Análisis de seguridad
`salud.yml`	Cada 6 h; manual	`GET /api/health` (disponibilidad)

Los reportes quedan en el *Job summary*, como artefactos y en `/reportes/` del sitio. Detalle en `docs/automatizaciones.md`.

8. Integraciones

Integración	Instalación
Extensión VS Code	`npm run build -w extensions/vscode` → `cd extensions/vscode && npx @vscode/vsce package --no-dependencies` → `code --install-extension atomreactchamber-vscode-2.0.0.vsix`; ajustar `atomreactchamber.apiUrl`
Servidor MCP	`npm run build -w integrations/mcp` → `claude mcp add atomreactchamber -e ARC_API_URL=http://localhost:8080 -- node integrations/mcp/dist/index.js` (o `.mcp.json`)
Skill de Claude Code	Se carga sola desde `.claude/skills/atomreactchamber`; ejecutar `scripts/arc.sh reaccion "H2 + O2"`
Subagente `tutor-quimi`	`.claude/agents/tutor-quimi.md`

Detalle en `docs/integraciones.md`.

9. Pruebas

Suite	Comando	Pruebas
Backend (integración con PostgreSQL real)	`DATABASE_URL=postgres://postgres@localhost:5432/arc npm test -w backend`	17
Motor + *solver* de niveles	`npm test -w frontend`	15
e2e + accesibilidad (axe, WCAG 2.2 AA)	`npm run e2e -w frontend` (primera vez: `npx playwright install chromium`)	7
Extensión VS Code	`npm test -w extensions/vscode`	19
Servidor MCP	`npm test -w integrations/mcp`	12

10. Solución de problemas

Síntoma	Causa probable	Solución
La API no arranca: “Configuración inválida → JWT_SECRET…”	Variable faltante o corta	Definir `JWT_SECRET` (≥ 32) y `DATABASE_URL`; cargar `backend/.env` con `set -a; . backend/.env; set +a`
`/api/health` responde `estado: degradado`, `db: error`	BD inaccesible	Revisar `DATABASE_URL`, que PostgreSQL esté activo y, en Neon, que la URL tenga `sslmode=require`
Error de SSL contra PostgreSQL local	TLS activado para un host no reconocido como local	`DB_SSL=disable`

Síntoma	Causa probable	Solución
La primera solicitud tarda ~1 minuto	Render Free suspende el servicio tras 15 min sin tráfico	Normal; `salud.yml` lo despierta cada 6 h; usar modo demo en clase
El navegador muestra error de CORS	`CORS_ORIGIN` no incluye la URL del sitio	Agregar la URL exacta (con `https://`)
"relation … does not exist"	Migraciones no aplicadas	`liquibase update`
Liquibase: "checksum changed" en un *changeset* de tabla	Se editó un *changeset* ya aplicado	Revertir la edición y crear un *changeset* nuevo; solo funciones, vistas y semillas son `runOnChange`
Las pruebas del backend fallan con `ECONNREFUSED`	Sin PostgreSQL local	Iniciar PostgreSQL 16, crear `arc`, migrar y exportar `DATABASE_URL`
`npm run e2e` falla al abrir el navegador	Falta Chromium de Playwright	`npx playwright install --with-deps chromium`
Registro de docente: "Código de docente incorrecto"	`DOCENTE_CODIGO` distinto	Usar el valor configurado en la API
Un usuario desactivado sigue usando la app	El JWT dura 7 días y `activo` solo se valida al iniciar sesión	Limitación conocida; rotar `JWT_SECRET` invalida todos los tokens
Un *workflow* dice "omitido: falta secret X"	*Secret* no configurado	Cargarlo (sección 6)
El sitio de Pages no se publica	Pages no habilitado	*Settings → Pages → Source: GitHub Actions*

11. Seguridad operativa

- Nunca versionar `.env`, `terraform.tfvars` ni respaldos (están en `.gitignore`).
- Cambiar `DOCENTE_CODIGO` antes del piloto y compartirlo solo con docentes.
- Revisar las alertas de CodeQL y los PR de Dependabot cada semana.
- ⚠️ **Antes de un piloto con estudiantes menores de 14 años**, la IE debe recoger el consentimiento de los padres fuera de la aplicación (el flujo dentro de la aplicación es trabajo futuro).